



# 12주차 과제

|             |             |
|-------------|-------------|
| ⚙ Status    | In progress |
| 📁 Task type |             |

## 1. TabNet

### 1. Abstract — 해결하려는 문제

- 기존 tabular 모델:
  - **Tree 기반(XGBoost, LightGBM)** dominate
  - 딥러닝은 성능/구조적으로 부적합
- 문제:
  - DNN → feature 중요도 반영 못함
  - overparameterization
  - interpretability 부족

- 해결:

👉 feature를 단계적으로 선택하는 딥러닝 모델

- 목표:

👉 성능 + 해석 가능성 + 효율성 동시에 달성

### 2. Introduction — 연구 동기

- 현실 데이터 대부분:

## 👉 tabular data (표 데이터)

---

- 기존 상황:
  - Tree:
    - ✓ 성능 좋음
    - ✓ 해석 가능
  - DNN:
    - ✗ tabular에 약함
- 

## 👉 핵심 아이디어:

- 딥러닝을

## 👉 “feature 선택 + 단계적 판단 구조”로 바꾸자

---

- 이유:
  - 중요한 feature만 써야 효율적
  - 사람도 단계적으로 판단함
- 

## 3. Related Works — 기존 한계

| 방법                        | 문제                   |
|---------------------------|----------------------|
| Decision Tree             | end-to-end 학습 불가     |
| MLP                       | feature selection 없음 |
| Feature selection methods | 별도 모델 필요             |

---

## 👉 TabNet 차별점:

- **feature selection + prediction 통합**
  - instance-wise feature 선택
  - 하나의 모델로 해결
- 

## 4. Method — 모델 구조

## 구조

- Feature Transformer
- Attentive Transformer (핵심)
- Sequential Decision Steps

👉 전체 흐름

```
input → step1 → step2 → ... → output
```

## 핵심 Objective

👉 feature selection + prediction 동시에 학습

## 학습 방식

- mask 기반 feature 선택
- sparsemax 사용
- end-to-end gradient descent

## 5. 핵심 이론

### Feature Selection (핵심)

👉 각 step마다 feature 선택

```
selected_feature = mask * input
```

👉 의미:

- 중요한 feature만 사용
- 불필요한 feature 제거

## Sequential Learning

👉 여러 단계로 나눠서 판단

step1 → 일부 정보 처리

step2 → 추가 정보 반영

...

👉 사람처럼 생각:

- 한 번에 판단 ❌
- 나눠서 판단 ○

## Sparse Attention

👉 sparsemax 사용

- softmax → 다 사용
- sparsemax → 일부만 선택

## Interpretability

👉 mask 자체가 설명 역할

- 어떤 feature 썼는지 바로 알 수 있음

## 6. Experiments — 실험

### 데이터

- 다양한 tabular dataset
- synthetic + real-world

### 결과

- Tree 모델 대비:

👉 동등 또는 더 좋은 성능

- 예:

👉 Forest Cover dataset

- XGBoost: 89%
- TabNet: **96%**

---

## 특징

- instance-wise feature selection에서 강함
- parameter 효율성 높음

---

## 7. Discussion — 장단점

### 장점

- interpretability 가능
- feature selection 자동
- end-to-end 학습
- 높은 성능

---

### 단점

- 구조 복잡
- 학습 비용 증가
- 하이퍼파라미터 영향 있음

---

## 8. Conclusion

- 기존 방식:

👉 "모든 feature → 한 번에 처리"

- TabNet:

👉 "중요한 feature → 단계적으로 처리"

## 9. 기존 대비 차별점

| 항목               | 기존 DNN | TabNet |
|------------------|--------|--------|
| feature 사용       | 전체     | 선택적    |
| 구조               | 단일     | 단계적    |
| interpretability | 없음     | 있음     |
| 효율성              | 낮음     | 높음     |

## 10. 핵심 아이디어

👉 “중요한 feature만 선택해서 단계적으로 판단한다”

## 11. 코드 구조 (핵심)

```
# feature selection
mask = sparsemax(attention(prev))

# apply mask
x = mask * input

# feature transform
d, a = transformer(x)

# aggregate
output += relu(d)
```

## 🔥 최종 한 줄 요약

👉 TabNet = feature를 선택하면서 단계적으로 판단하는, 해석 가능한 tabular 딥러닝 모델

## 2. TabPFN (Tabular Foundation Model)

### 1. Abstract — 해결하려는 문제

- 기존 tabular 문제:
  - 데이터가 작음 (대부분 1만 샘플 이하)
  - 모델마다 따로 학습 필요
  - Tree 모델이 여전히 dominant

- 문제:
  - 딥러닝 → tabular에서 약함
  - 데이터 간 지식 공유 불가능
  - 작은 데이터에서 성능 낮음

- 해결:

👉 tabular용 foundation model (TabPFN)

- 목표:

👉 작은 데이터에서도 빠르고 강력한 예측

👉 학습 없이도 바로 적용 (inference만)

### 2. Introduction — 연구 동기

- 최근 흐름:

| 분야  | 변화                       |
|-----|--------------------------|
| CV  | handcrafted → CNN        |
| NLP | rule-based → Transformer |
| 게임  | heuristic → RL           |

👉 핵심 흐름:

“알고리즘을 직접 만드는 게 아니라, 학습으로 만든다”

- tabular의 특징:
- 데이터마다 의미 다름
- 구조 다양함
- 대부분 작은 데이터

👉 기존 문제:

- 모델을 데이터마다 새로 학습해야 함

👉 핵심 아이디어:

👉 “모델이 아니라 학습 알고리즘 자체를 학습하자”

### 3. Related Works — 기존 한계

| 방법               | 문제           |
|------------------|--------------|
| Tree (XGBoost 등) | transfer 불가능 |
| DNN              | 작은 데이터에서 약함  |
| 기존 PFN           | 확장성 부족       |

👉 TabPFN 차별점:

- dataset 단위로 학습
- transformer 기반
- in-context learning 적용

### 4. Method — 모델 구조

#### 구조

- Transformer 기반
- 2D attention (row + column)



👉 핵심 구조 (논문 Fig.1b, p.2)

- row 방향 attention → 샘플 간 관계
- column 방향 attention → feature 간 관계

## 핵심 Objective

👉 masked target prediction

```
input: (X_train, y_train, X_test)
output: y_test
```

👉 특징:

- train + test를 **같이 넣음**
- 한 번의 forward로 예측

## 학습 방식

1. synthetic dataset 생성
2. 일부 label masking
3. transformer가 예측

👉 핵심:

👉 수백만 개 synthetic dataset으로 사전학습

## 5. 핵심 이론

### In-Context Learning (ICL)

👉 LLM과 동일 개념

```
모델 = 학습된 알고리즘
데이터 = prompt
```

👉 의미:

- 모델이 "학습 방법 자체"를 배움
- inference 시:

👉 학습 + 예측 동시에 수행

## 기존 vs TabPFN

| 기존 ML           | TabPFN   |
|-----------------|----------|
| train → predict | 한 번에     |
| 데이터마다 학습        | 학습 필요 없음 |
| 모델 고정           | 알고리즘 학습  |

## Synthetic Data Learning

👉 핵심 아이디어

수백만 가짜 데이터 생성 → 모델 학습

👉 이유:

- 실제 데이터 다양성 커버
- 다양한 문제 자동 학습

👉 논문 설명 (p.3):

- causal model 기반 데이터 생성
- missing value, noise, categorical 포함

## Bayesian 해석

👉 TabPFN은:

$p(y_{\text{test}} \mid X_{\text{train}}, y_{\text{train}}, X_{\text{test}})$

을 근사함

👉 즉:

👉 “베이지안 추론을 neural network로 근사”

---

## 6. Experiments — 실험

---

### 데이터

- OpenML, AutoML benchmark
  - 최대:
    - 10,000 samples
    - 500 features
- 

### 결과

👉 TabPFN vs CatBoost

- classification:
    - +0.13 성능 향상
  - regression:
    - +0.09 향상
- 

👉 가장 충격적인 결과:

- TabPFN:
    - **2.8초**
  - 기존 모델:
    - **4시간 튜닝**
- 

👉 성능 더 좋음

---

👉 논문 핵심 문장:

| “2.8초 만에 4시간 튜닝한 모델보다 좋다”

---

## 특징

- hyperparameter tuning 거의 필요 없음
  - small data에서 특히 강함
- 

## 7. Discussion — 장단점

---

### 장점

- small data에서 압도적 성능
  - 학습 없이 바로 사용
  - 매우 빠름
  - uncertainty까지 예측 가능
- 

### 단점

- 큰 데이터에서는 제한 있음 (~10k)
  - synthetic prior에 의존
  - 메모리/구조 복잡
- 

## 8. Conclusion

- 기존:

👉 "데이터마다 모델 학습"

---

- TabPFN:

👉 "하나의 모델로 모든 데이터 해결"

---

## 9. 기존 대비 차별점

| 항목    | 기존 ML    | TabPFN |
|-------|----------|--------|
| 학습 방식 | dataset별 | 사전학습   |

| 항목             | 기존 ML       | TabPFN       |
|----------------|-------------|--------------|
| inference      | prediction만 | 학습+예측        |
| 데이터            | 실제          | synthetic 기반 |
| generalization | 낮음          | 높음           |

## 10. 핵심 아이디어

👉 “모델이 아니라 학습 알고리즘 자체를 학습한다”

## 11. 코드 구조 (핵심)

```
# input
X_train, y_train, X_test

# model (pretrained)
model = TabPFN()

# inference = training + prediction
y_pred = model(X_train, y_train, X_test)
```

## 🔥 최종 한 줄 요약

👉 TabPFN = 수백만 synthetic 데이터로 학습된 “학습 알고리즘 모델”로, 작은 tabular 데이터에서 학습 없이 바로 예측하는 foundation model

## 🔥 진짜 핵심 차이 (TabNet vs TabPFN)

| 모델     | 핵심         |
|--------|------------|
| TabNet | feature 선택 |
| TabPFN | 학습 자체를 학습  |